



# Deep Learning

## Lecture 9.1: Point Cloud Networks

# Today's Learning Objectives

**Be able to answer:**

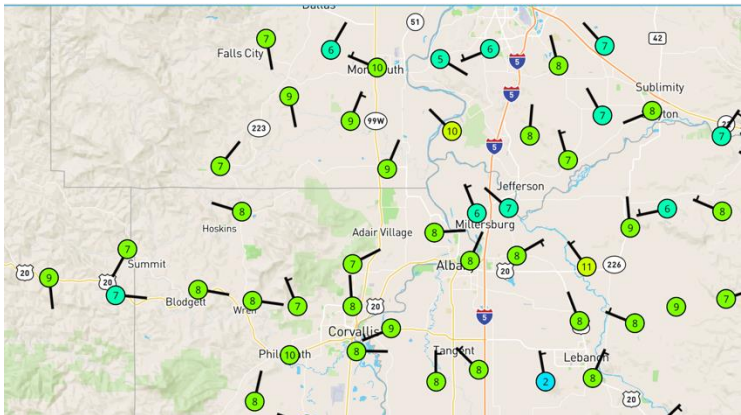
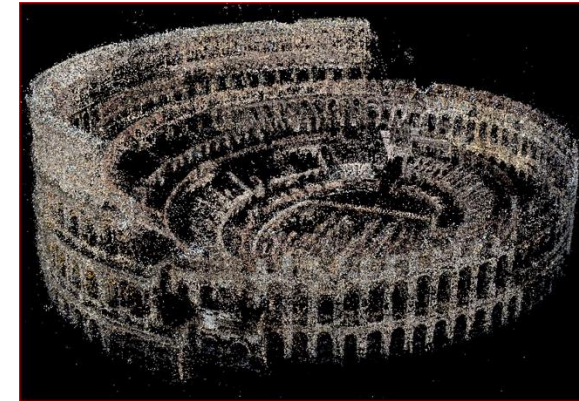
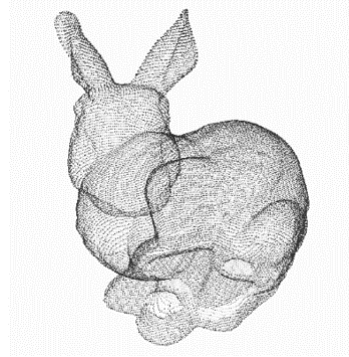
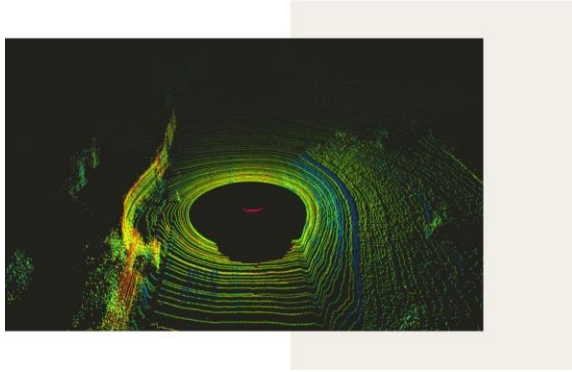
- What is a point cloud?
- What is a sparse convolutional network?
- What is point convolution?
- What is a point transformer?
- What are some applications of point cloud networks?





# What is a point cloud?

An unordered set of points in a low-dimensional Euclidean space



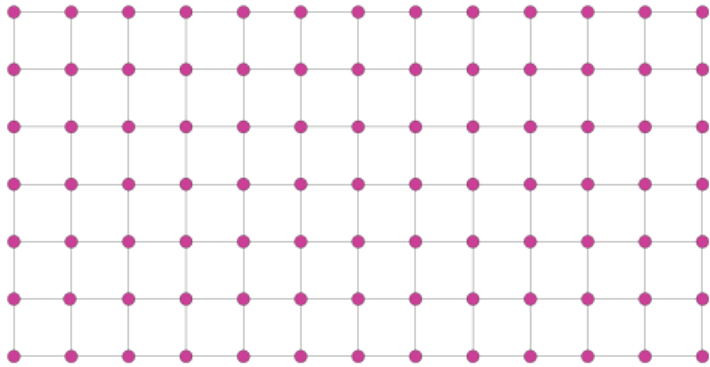
Why do we use the point cloud representation? In cases it costs too much to model the entire space (sparse)

And/or when the geometry of the data matters a lot



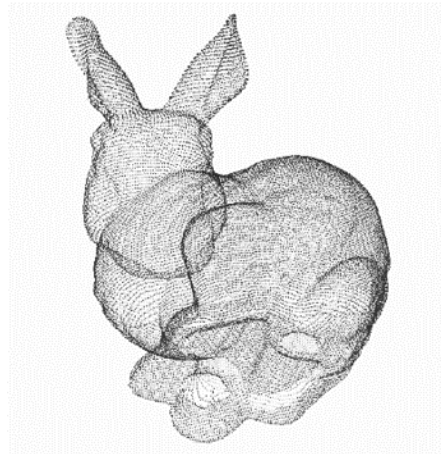
# Different types of spatial data

Images



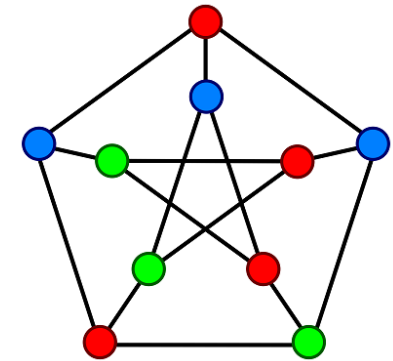
A regular lattice in a low-dimensional (2D/3D) **Euclidean space**

Point Clouds



An arbitrary set of points in a low-dimensional **Euclidean space**

Graphs



Connected nodes **not necessarily** in a low-dimensional Euclidean space

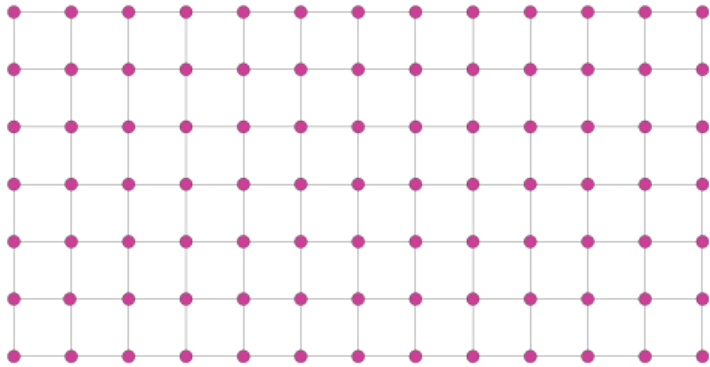
Euclidean space geometry shared by both images and point clouds

(In many graph networks nowadays, people utilize Euclidean position embeddings, essentially making them point cloud approaches)

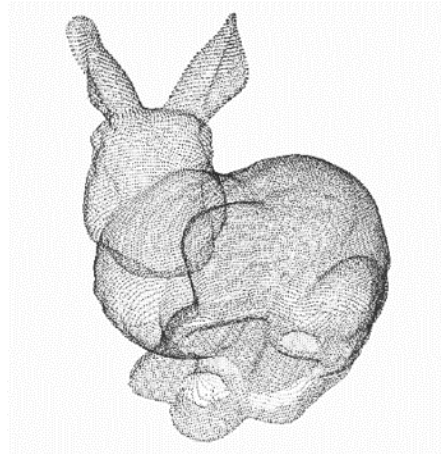


# What do we mean by Euclidean space geometry?

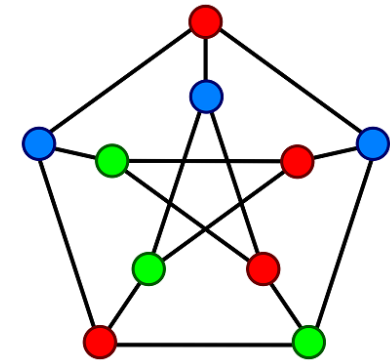
Images



Point Clouds



Graphs



1. Closeness (For locating a neighborhood)
2. Orthonormal coordinate system (e.g.  $(x,y)$  or  $(x,y,z)$ )

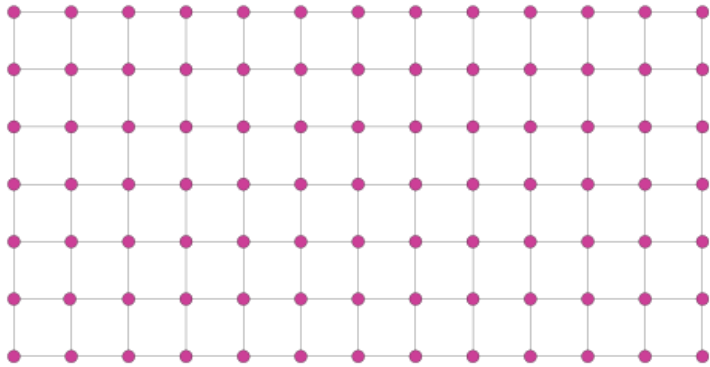
Can define closeness by connectivity  
No coordinate system

Why is coordinate system useful? Each dimension could have different characteristics!  
E.g. there is gravity in the  $z$  dimension



# How to define a point cloud neighborhood?

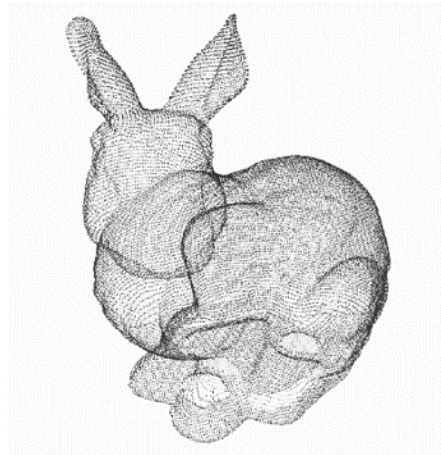
Images



3x3, 5x5, 7x7 etc.

In 3D images: 3x3x3, 5x5x5, 7x7x7

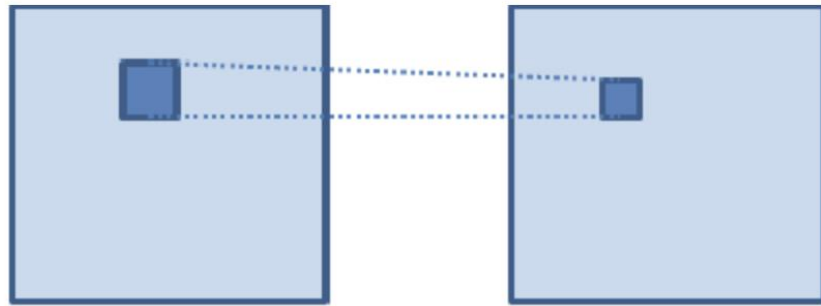
Point Clouds



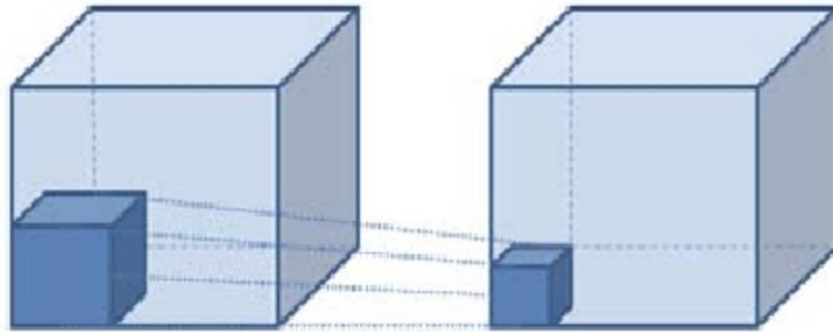
3x3x3, or kNN,  
or  $\epsilon$ -ball, or  
space-filling curves



# Let's start with the simplest one: 3x3x3 type neighborhoods



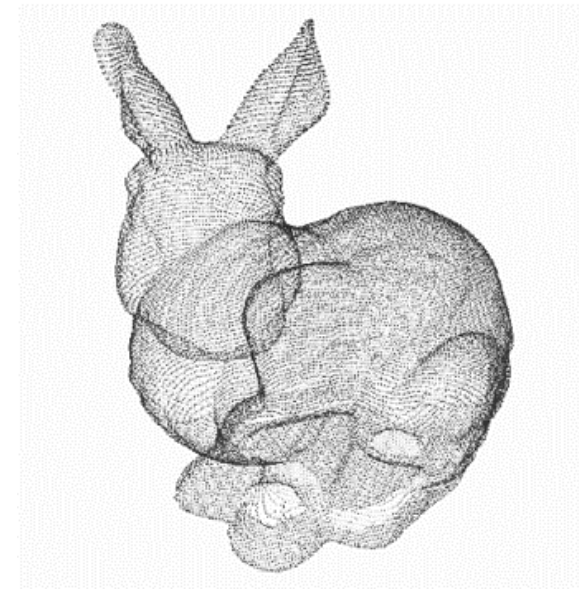
(a) 2D convolution



(b) 3D convolution

## 3. Comparison of (a) 2D convolution and (b)

Can we use this on  
a 3D point cloud?



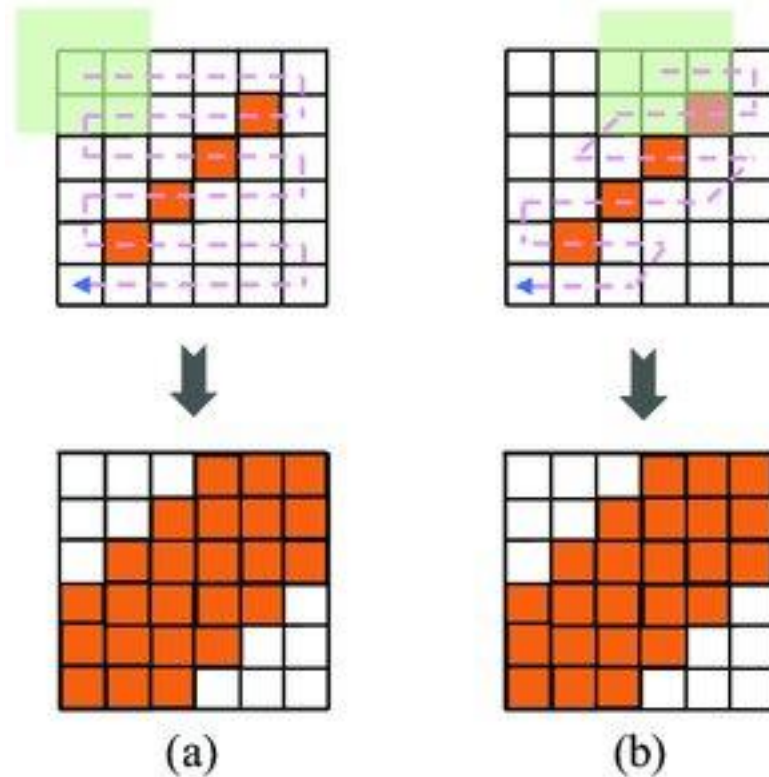


# Dense Convolution and Sparse Convolution

## Dense Convolution

We lose sparsity after a while!

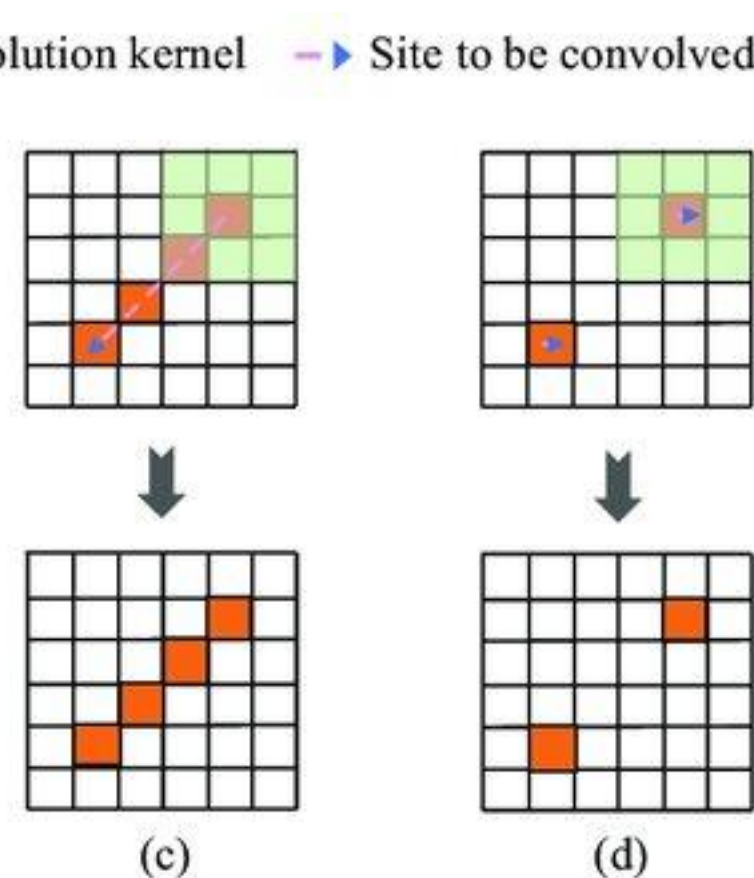
■ Active site    □ Inactive site    ■ The convolution kernel    ➡ Site to be convolved



In 3D, this quickly grows the amount of active sites

## Sparse Convolution

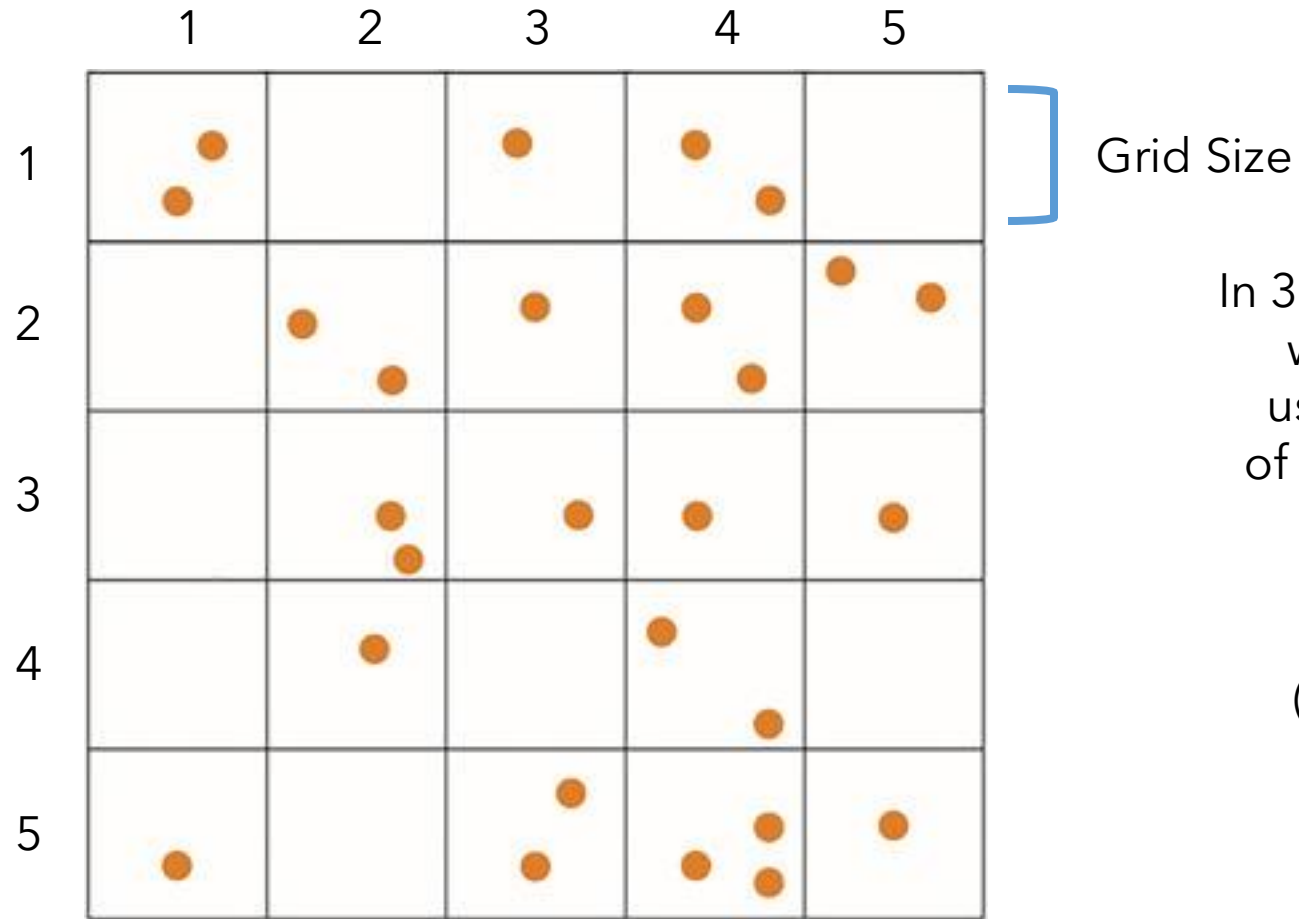
If input is 0, output is 0



Keeps the sparsity of the input



Bring the real-valued coordinates to integer values by quantization



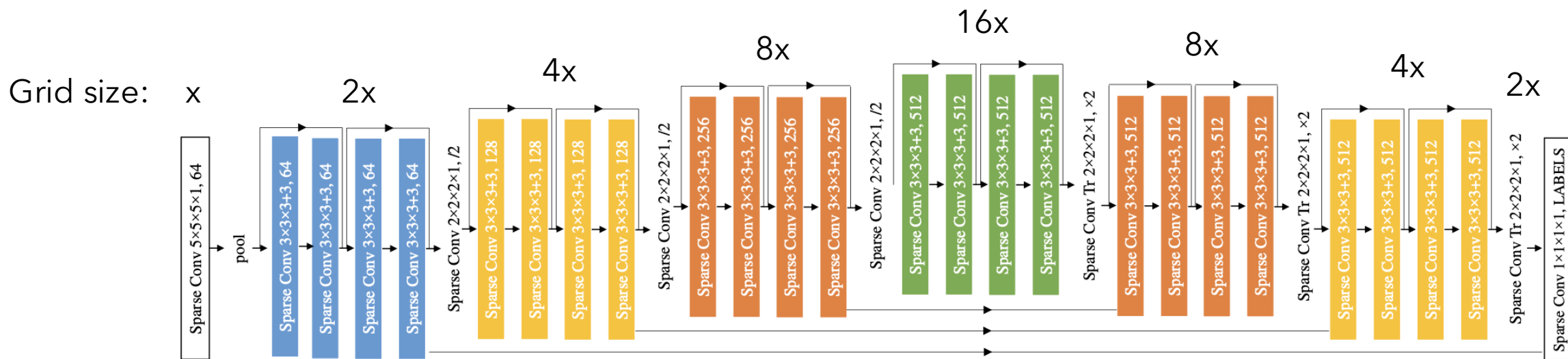
In 3D, since we are often dealing with the physical world, we use grid sizes in metric units of length, e.g. 2cm, 5cm, 10cm

With 2x downsampling (e.g. stride-2 convolution), the grid size will **double**



# UNet Structure for 3D Point Cloud Segmentation with Sparse Convolutions

For 3D point clouds, same idea as 2D UNet!



Almost everything can be plainly extended  
e.g. strided convolution with stride 2 or 1/2



# Some Semantic Segmentation Results on Point Clouds

ScanNet dataset:

1500 Room Scans  
with 3D annotations  
and all the 3D information



Figure 7: Visualization of Scannet predictions. From the top, a 3D input pointcloud, a network prediction, and the ground-truth.



# Many Applications Allowed

## 3D Object Detection

Image cf. A. Lang et al. PointPillars: Fast Encoders for Object Detection from Point Clouds. CVPR 2019



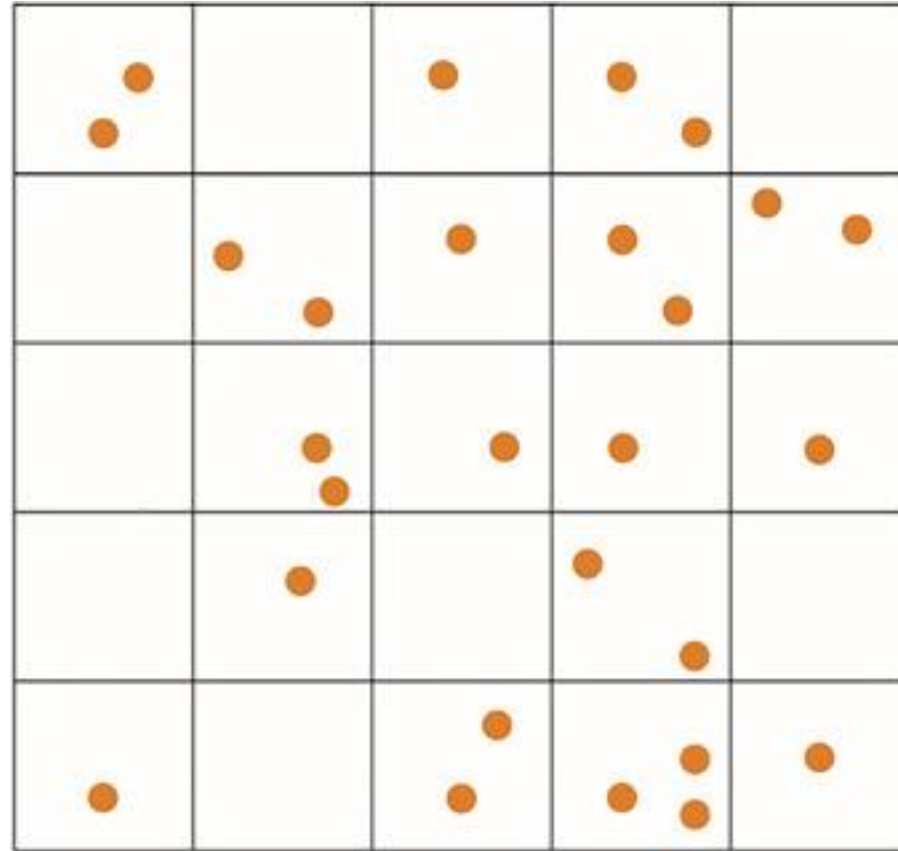
## 3D Instance Segmentation

Image cf. J. Schult et al. Mask3D: Mask Transformer for 3D Semantic Instance Segmentation. ICRA 2023





# When does sparse convolution perform well?



- When point density is relatively uniform  
(quantization would work well, and all points have a similar amount of neighbors)
- When the output is also required at a fixed resolution (or can be easily upsampled from it)

# Today's Learning Objectives

Be able to answer:

- ~~What is a point cloud?~~
- ~~What is a sparse convolutional network?~~
- What is point convolution?
- What is a point transformer?
- What are some applications of point cloud networks?

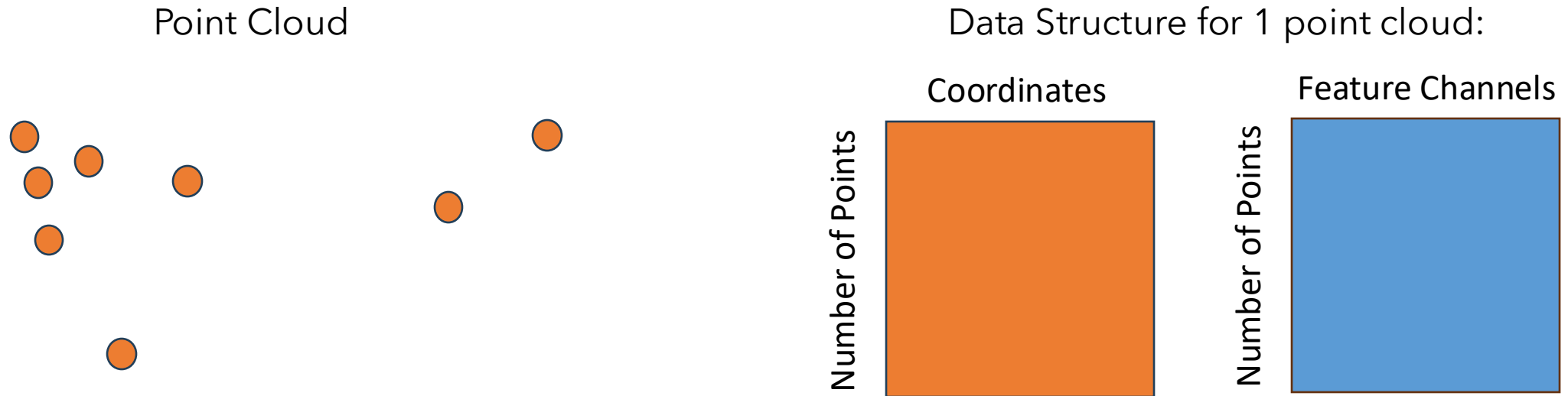




# Are there some other ideas to obtain a point cloud network?

We do not have to make the assumption that point density is relatively even

And directly work on “point-based” approaches



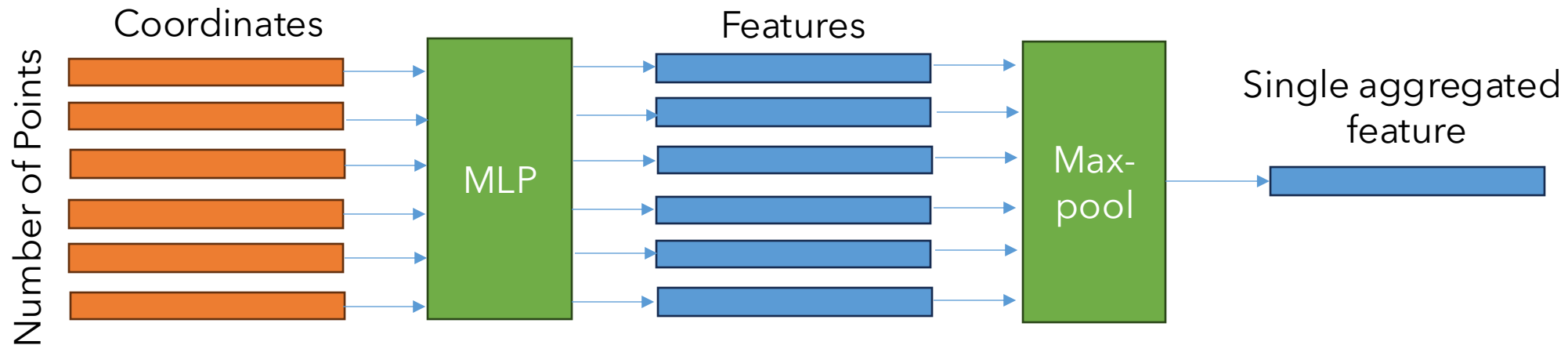
Difficulty: How to make a permutation-invariant network that does not change its behavior, when the input ordering is permuted?

Fully-connected networks surely won't do the job!



Simple idea: just perform the same operations on every point as different examples

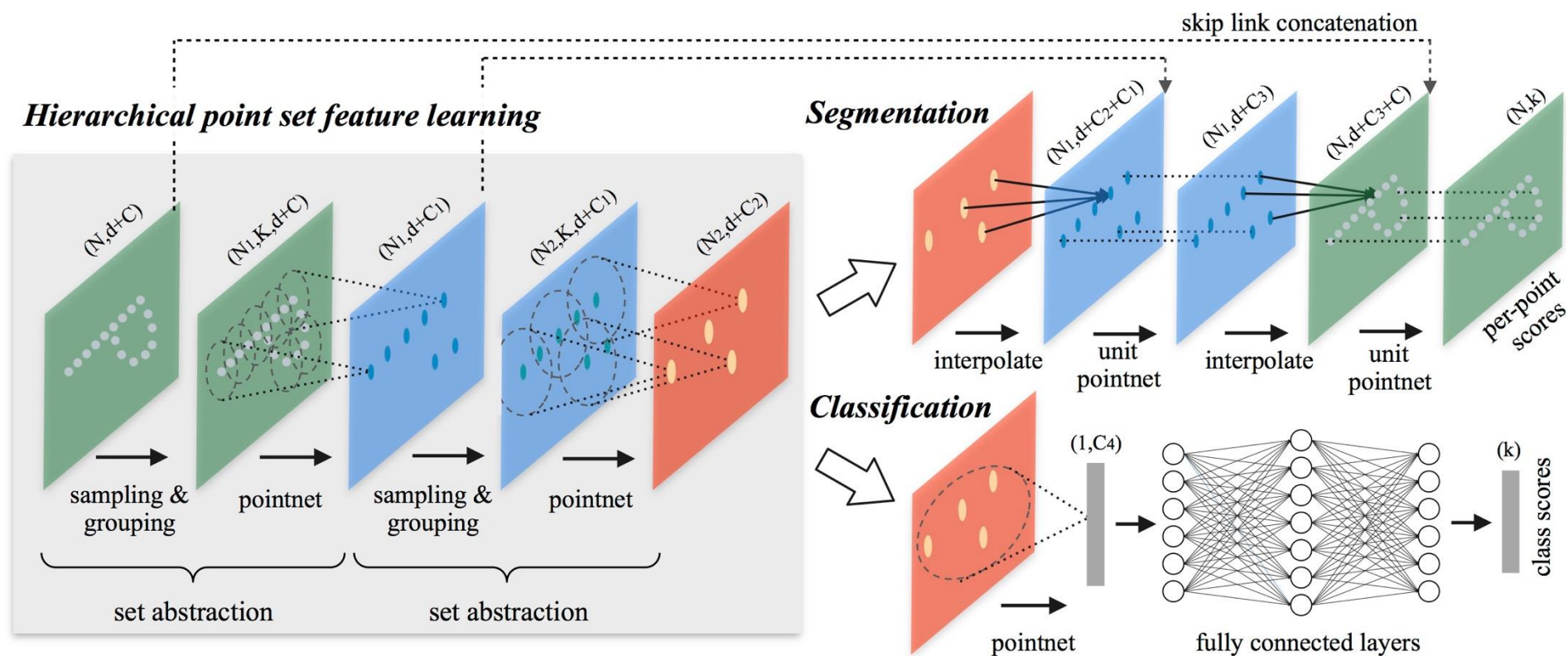
When we want to aggregate features, just perform max-pooling



This felt a bit crude in crushing all point features into a single feature,  
any simple improvements?



Main difference is to add grouping (with k-nearest neighbors) and subsampling  
(More similar to a normal computer vision pipeline)  
PointNet is applied within each neighborhood  
max-pooling is done with the kNN neighborhood

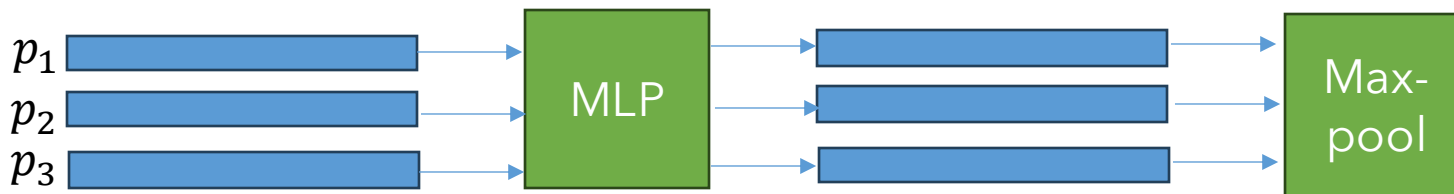




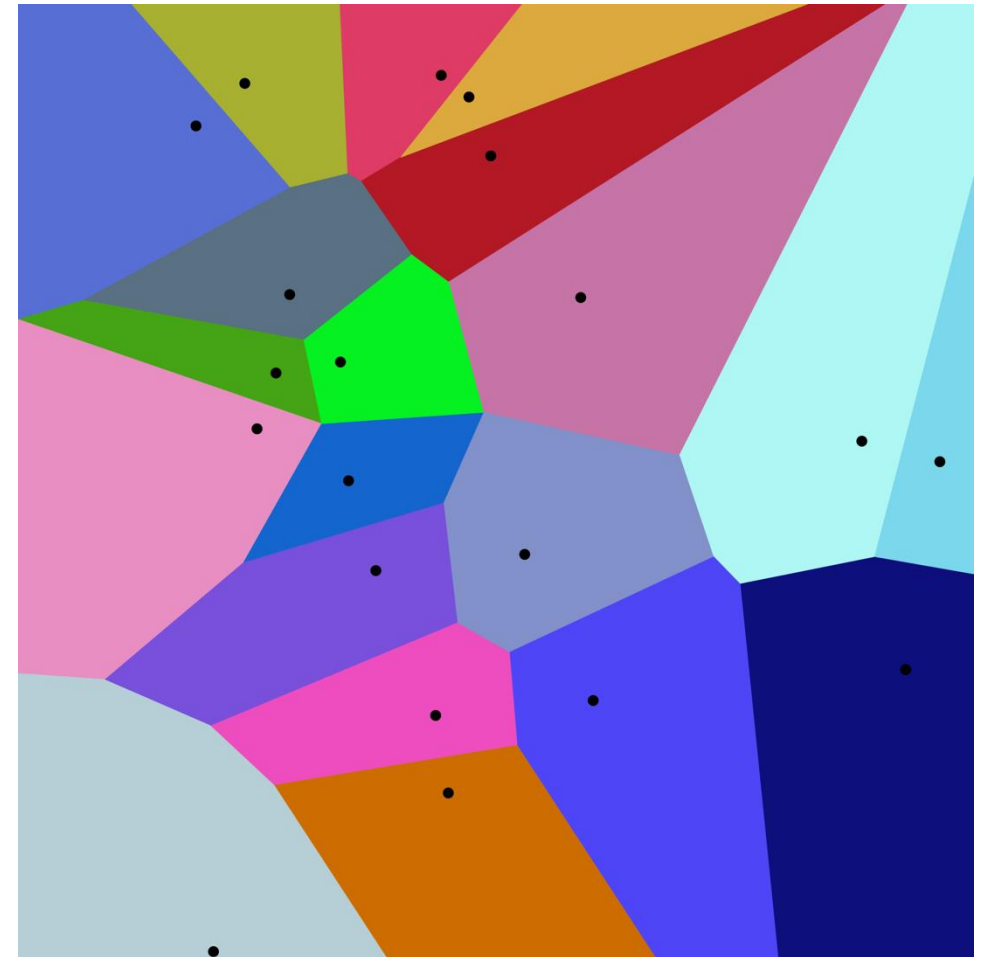
# What functions are PointNet++ realizing in the Euclidean space?

Treat PointNet++ as defining a function in the Euclidean space

If two points have the same set of neighbors,  
their output is the same



$$f(p_1) = f(p_2) = f(p_3)$$





# What if I don't think piecewise-constant functions are enough?

## Rethink Convolution

Convolution used to be  
defined in a continuous space

Since when everybody  
started to think convolution  
as 3x3 or 7x7 kernels?

Can we use this  
continuous definition  
to bring convolution back  
to irregular point clouds?



# From Continuous to Point Cloud Convolutions

-   $W$ : Weight kernel
-   $X$ : Input features
-   $p$ : Input Coordinates

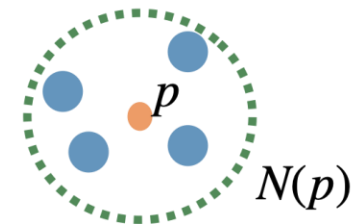
## Continuous Convolution

$$\text{Conv}(W, X)_p = \int_{\mathbb{R}^d} W(p_i - p) X_{p_i} dp_i$$

**Idea #1:** Discretize to points in neighborhood  $N(p)$   
and learn  $W$  (Independently developed by many groups)

## Discrete Convolution

$$\text{Conv}(W, X)_p = \sum_{p_i \in N(p)} W(p_i - p) X_{p_i}$$



(Monte Carlo Estimation, very common in mathematics)

(Underlying assumption: The weight kernel goes to close to 0 outside the neighborhood,  
a similar assumption to 3x3 kernels)



# PointConv: More efficient point convolution

  $W$ : Weight kernel  
  $X$ : Input features  
  $p$ : Input Coordinates

## Discrete Convolution

$$\text{Conv}(W, X)_p = \sum_{p_i \in N(p)} W(p_i - p) X_{p_i}$$



**Idea #2:** Assuming  $W = W_2 M(p_i - p)$ ,  $M$  is an MLP

## (Efficient) PointConv

$$\text{Conv}(W, X)_p = W_2 \sum_{p_i \in N(p)} M(p_i - p) \otimes X_{p_i}$$

1x1 conv  
(e.g.  $4C_{in} \rightarrow 64$ )

MLP with output  
channels  $C_{mid} = 4-16$

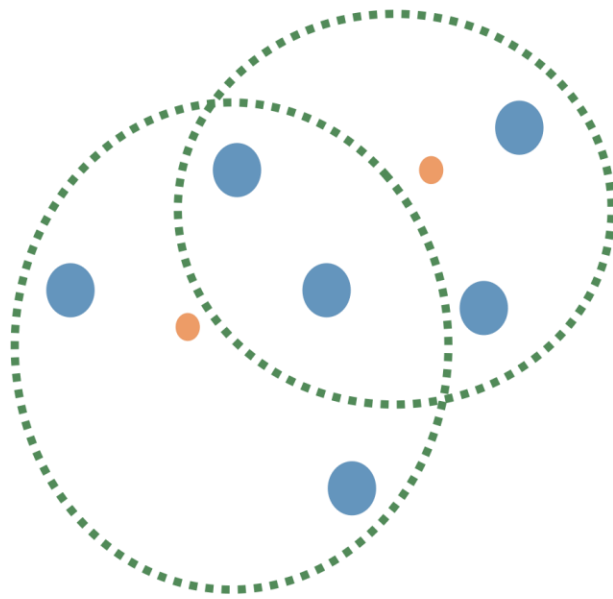
$\otimes$  denotes the  
 Kronecker product:  
 $\mathbb{R}^{C_{in}} \otimes \mathbb{R}^{C_{mid}} \mapsto \mathbb{R}^{C_{in} C_{mid}}$



$$\text{Conv}(W, X)_p = W_2 \sum_{p_i \in N(p)} M(p_i - p) \otimes X_{p_i}$$

PointConv is **well-defined** anywhere in  $\mathbb{R}^d$

It allows ***learnable interpolation***



● : Points with features

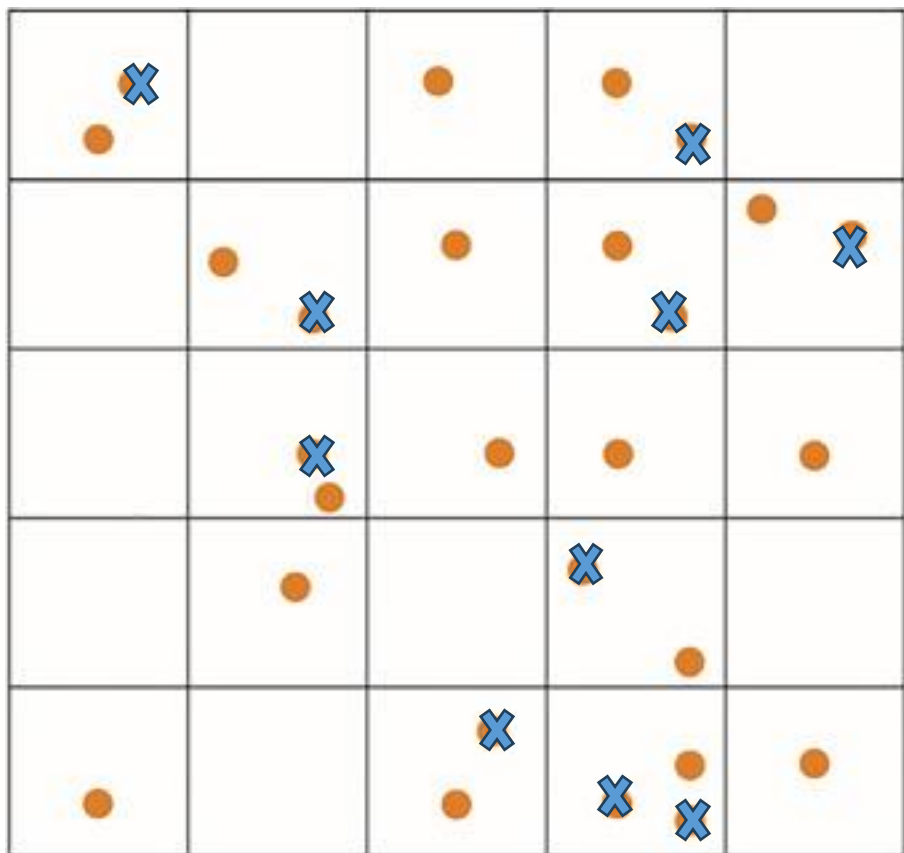
● : Query points with no features

Think about e.g. splines that defines a general nonlinear function on  $\mathbb{R}^d$



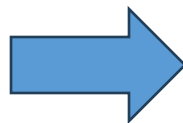
# Grid Downsampling

First level

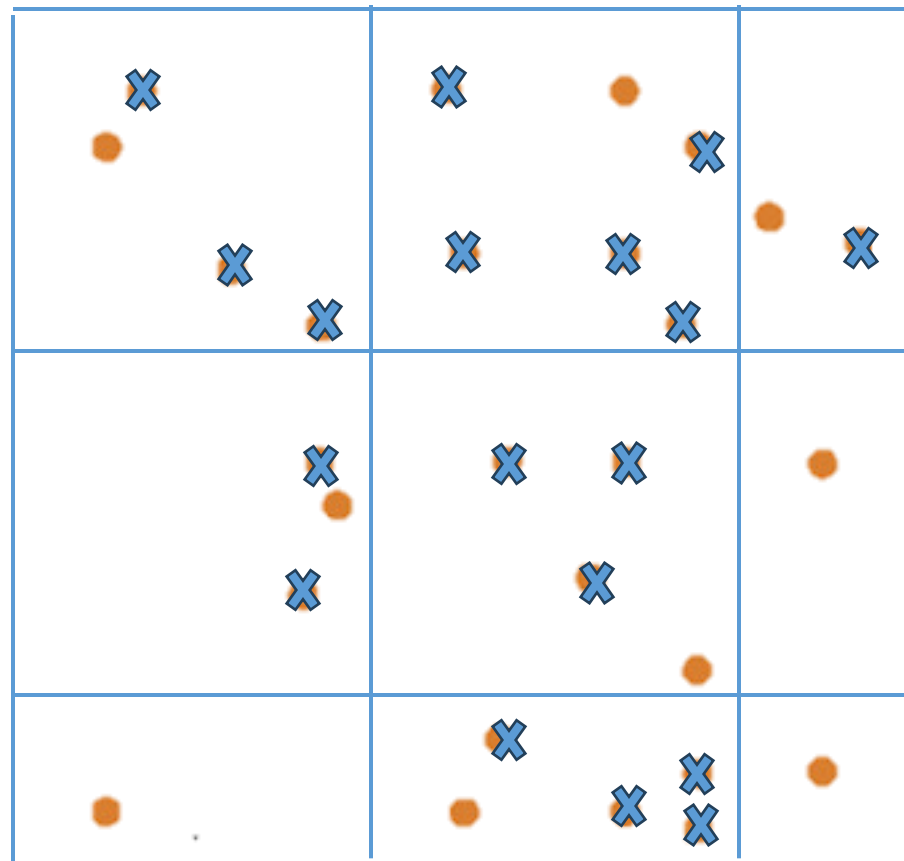


● : Sampled points

Grid  
size  
increase  
by 2x

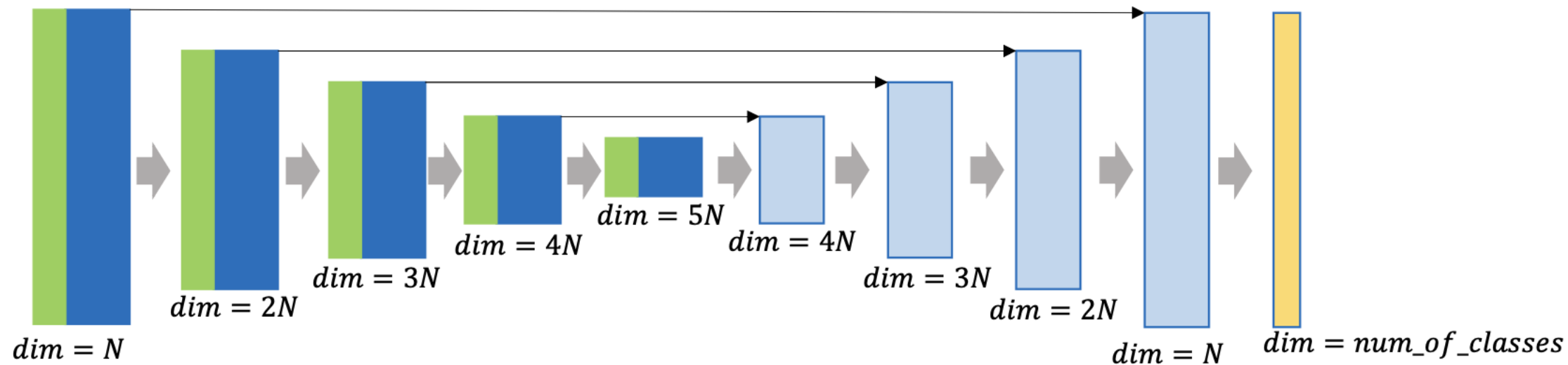


Second level





# PointConv U-Net Structure



Grid Subsampling



PointConv



PointConv Interpolation



Fully-Connected



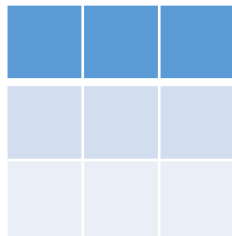
# Relative Position Vector and the difference between PointConv and 3x3

$$\text{Conv}(W, X)_p = W_2 \sum_{p_i \in N(p)} M(p_i - p) \otimes X_{p_i}$$

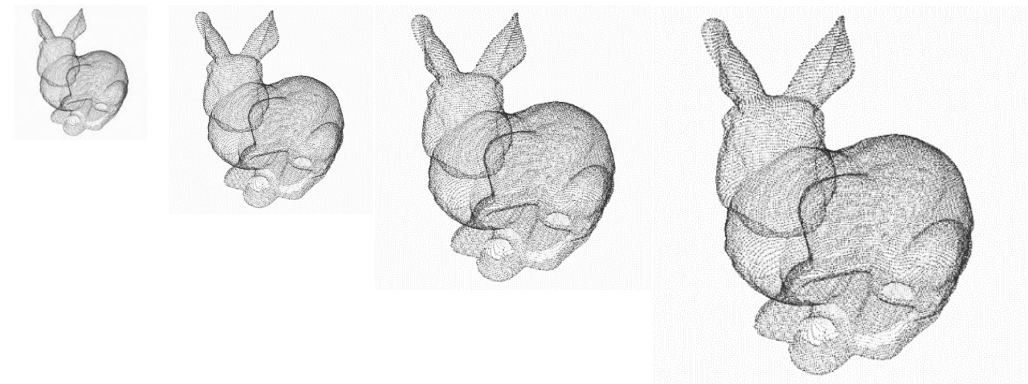
Think again about the relative position vector

Convolution is defined as a function on the relative position vector,  
multiplied with the input feature

This may be able to do  
something the 3x3 cannot do



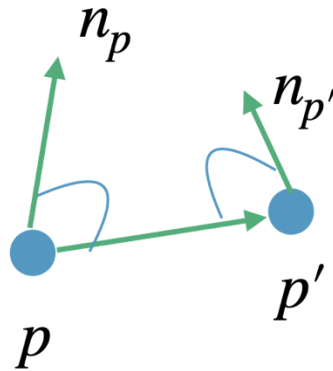
e.g. Scale-invariance



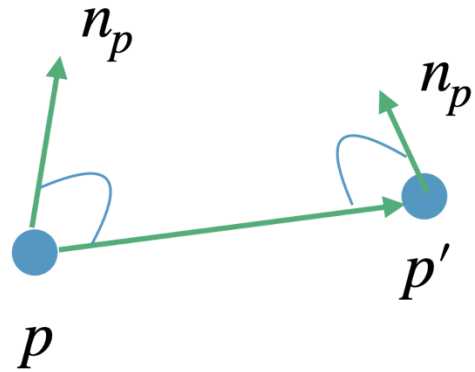
Relative position between points is just multiplied by a constant  
Can we get a function to ignore that constant?



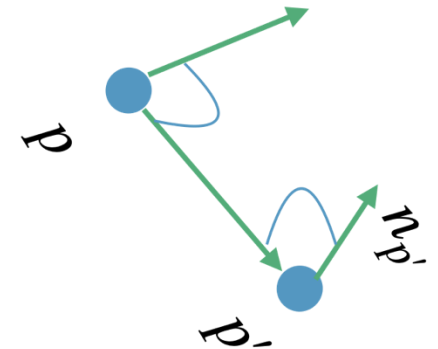
- Define a local basis to incorporate invariance in the weights



$n_p$ : the surface normal vector at  $p$



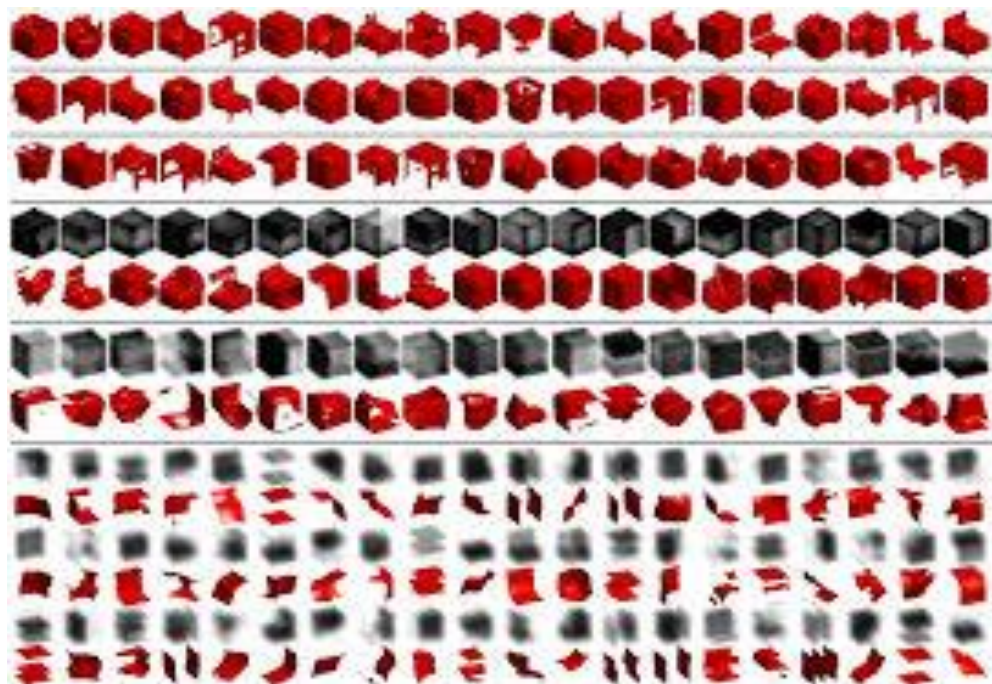
Global scaling:  
angles are invariant  
 $\|p - p_i\|^2$  is equivariant



Global rotation:  
angles are invariant  
 $\|p - p_i\|^2$  is invariant



## ModelNet40: Classify CAD Model Point Clouds



Common Approaches Require Rotation Augmentations

VI-PointConv Does Not

| Method                     | SO3/SO3 | none/SO3    |
|----------------------------|---------|-------------|
| PointNet(with T-Net) [42]  | 79.9    | -           |
| PointNet++ [44]            | 80.6    | -           |
| DGCNN(with T-Net) [65]     | 84.4    | -           |
| RS-CNN [37]                | 82.6    | -           |
| SpiderCNN(with T-Net) [73] | 78.7    | -           |
| RIConv [78]                | 86.4    | -           |
| SphericalCNN [8]           | 86.9    | -           |
| SRI-Net [54]               | 87.0    | -           |
| Triangle-Net [70]          | 86.7    | -           |
| SPH-Net [41]               | 87.6    | -           |
| RTN+DGCNN [12]             | 86.5    | -           |
| ClusterNet [4]             | 87.1    | -           |
| PointConv [66]             | -       | 61.0        |
| VI-PointConv (ours)        | -       | <b>88.2</b> |

Table 1. Comparison on ModelNet40 with Data SO(3) for 3D point cloud classification. SO3/SO3 indicates the model is trained & tested with SO(3) rotations, and none/SO3 indicate the model is trained without SO3 augmentations & tested with SO(3) rotations.



- Encode them as priors so that the network can learn when to use them

$$\text{Conv}(W, X)_p = W_2 \sum_{p_i \in N(p)} M(V(p_i - p), p_i - p) \otimes X_{p_i}$$

- where  $V(p_i - p)$  is a viewpoint-invariant (VI) coordinate transform

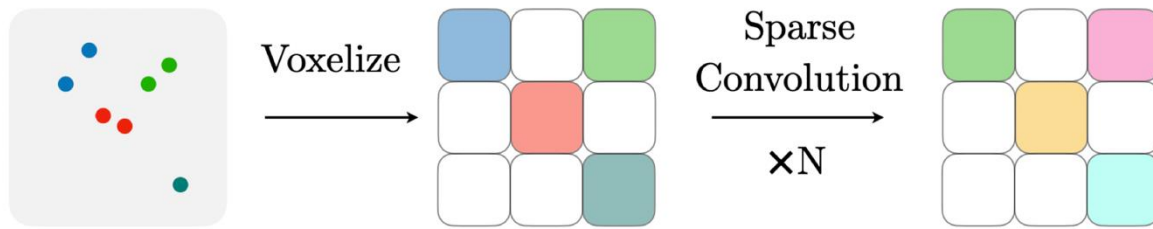
**ScanNet Semantic Segmentation Validation mIoU on PointConv w. Different Input to M**

| Input to M                 | 100k points training/<br>testing | 100k points training<br>60k points testing | 100k points training<br>40k points testing | 100k points training<br>20k points testing |
|----------------------------|----------------------------------|--------------------------------------------|--------------------------------------------|--------------------------------------------|
| $[V(p - p_i), p - p_i]$    | <b>68.2</b>                      | <b>64.4</b>                                | <b>63.0</b>                                | <b>57.6</b>                                |
| $p - p_i$                  | 63.3                             | 60.7                                       | 59.7                                       | 55.0                                       |
| $[n(p) - n(p_i), p - p_i]$ | 60.2                             | 57.5                                       | 56.8                                       | 54.8                                       |
| $n(p) - n(p_i)$            | 53.0                             | 50.6                                       | 50.2                                       | 47.6                                       |

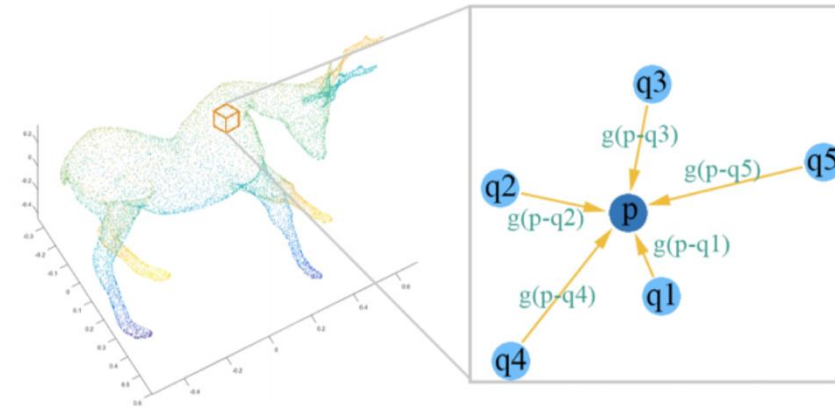


# Voxel Representation vs. Point Representation

Sparse Conv



PointConv



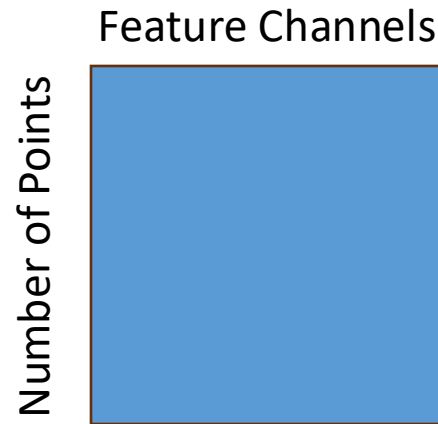
## Voxel Representation

- ✓ Ability to use regular 3x3x3 neighborhoods
- ✓ Convolution (SparseConv)
- ✗ No easy incorporation of invariances

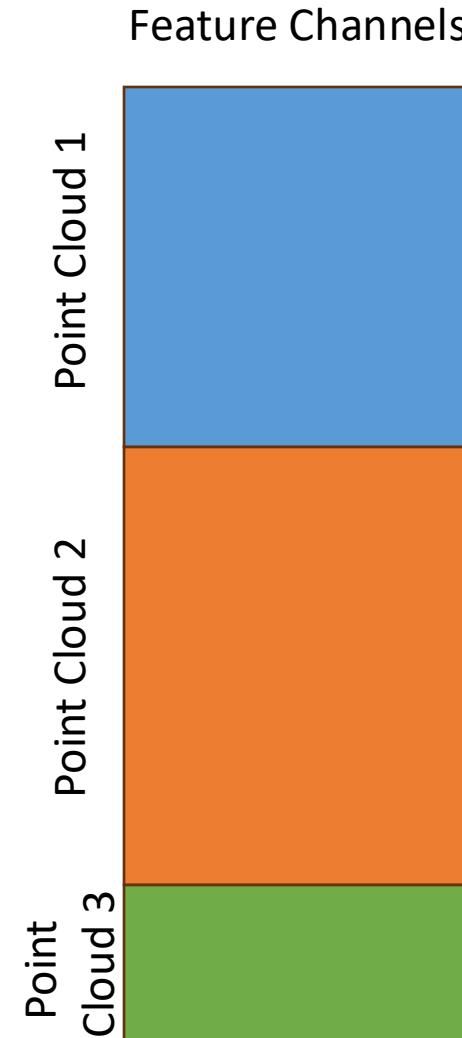
## Point Representation

- ✗ kNN neighborhood
- ✓ PointConv
- ✓ Incorporating rotation and scale invariance with VI-PointConv

Nice for 1 point cloud



What about point clouds that have different number of points?



# Today's Learning Objectives

Be able to answer:

- ~~What is a point cloud?~~
- ~~What is a sparse convolutional network?~~
- ~~What is point convolution?~~
- What is a point transformer?
- What are some applications of point cloud networks?

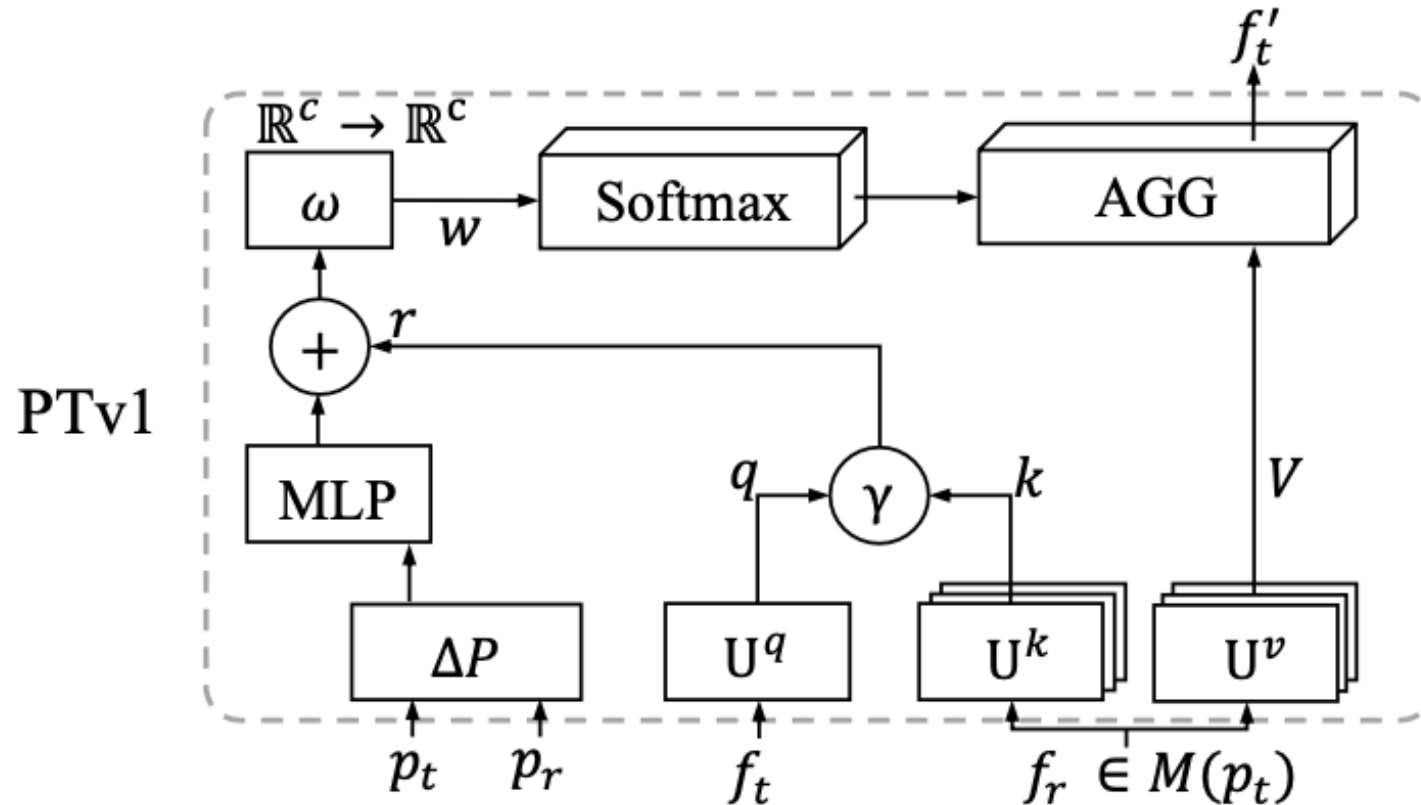




## Point Transformer

Transformer on point clouds  
Local Attention on neighborhoods  
Relative Position Embedding

Hengshuang Zhao<sup>1,2</sup> Li Jiang<sup>3</sup> Jiaya Jia<sup>3</sup> Philip Torr<sup>1</sup> Vladlen Koltun<sup>4</sup>  
<sup>1</sup>University of Oxford    <sup>2</sup>The University of Hong Kong  
<sup>3</sup>The Chinese University of Hong Kong    <sup>4</sup>Intel Labs

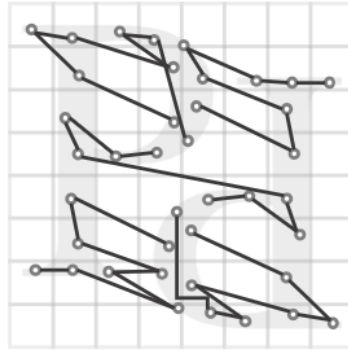
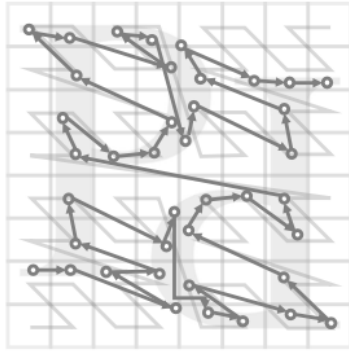
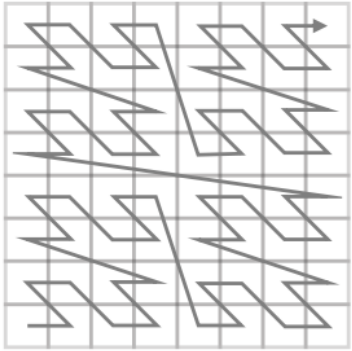




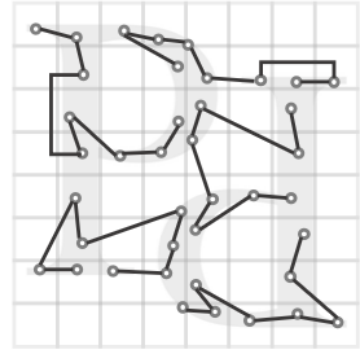
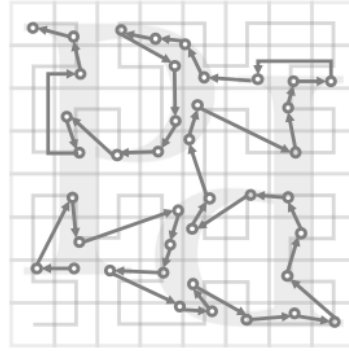
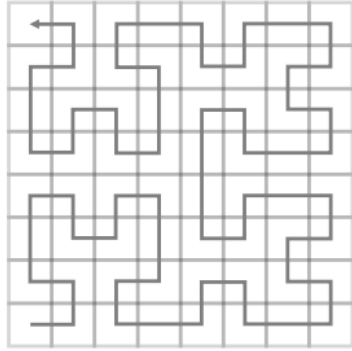
# Another Neighborhood Search Approach: Space Filling Curves

Instead of doing k-nearest neighbor search (costly), just sort the point clouds!  
Alternate the space-filling curves in different layers for better performance

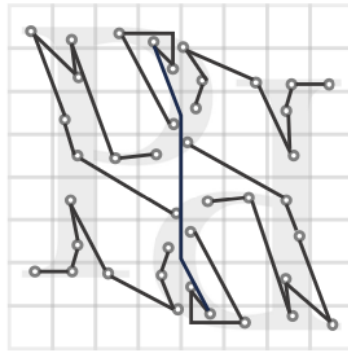
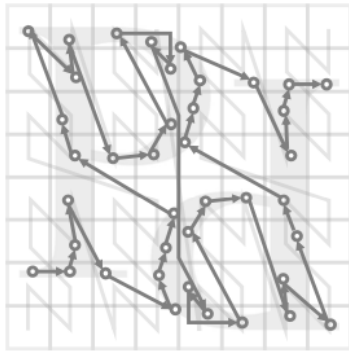
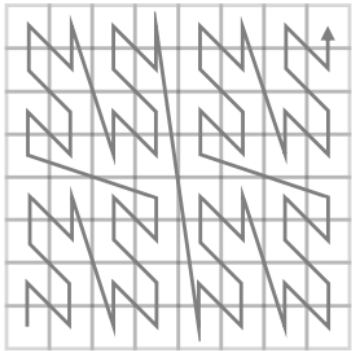
(a) Z-order



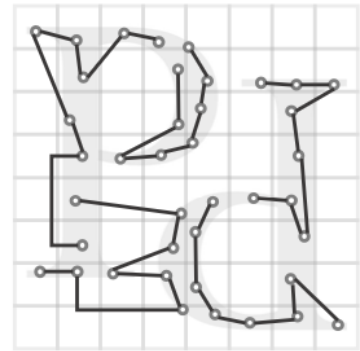
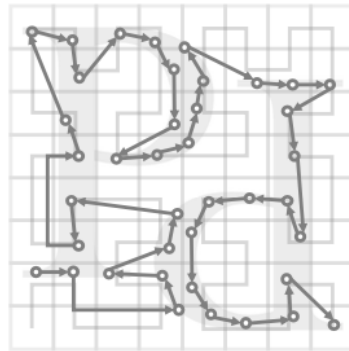
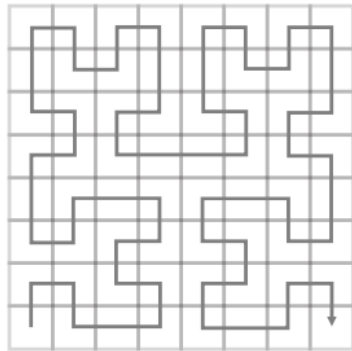
(b) Hilbert



(c) Trans Z-order



(d) Trans Hilbert



**Figure 3. Point cloud serialization.** We show the four patterns of serialization with a triplet visualization. For each triplet, we show the space-filling curve for serialization (left), point cloud serialization var sorting order within the space-filling curve (middle), and grouped patches of the serialized point cloud for local attention (right). Shifting across the four serialization patterns allows the attention mechanism to capture various spatial relationships and contexts, leading to an improvement in model accuracy and generalization capacity.

- Use space-filling curves to locate neighbors – rotate over different curves for each time
- Mix with sparse convolution – 1 sparse convolution layer followed by 1 transformer layer
- Layer normalization instead of batch normalization for transformer layers
- Grid subsampling (with BN for the grid subsampling layer)

Point  
Convolution →

Sparse  
Convolution →

| Methods                      | Year | Val  | Test |
|------------------------------|------|------|------|
| ○ PointNet++ [59]            | 2017 | 53.5 | 55.7 |
| ○ 3DMV [15]                  | 2018 | -    | 48.4 |
| ○ PointCNN [44]              | 2018 | -    | 45.8 |
| ○ SparseConvNet [23]         | 2018 | 69.3 | 72.5 |
| ○ PanopticFusion [52]        | 2019 | -    | 52.9 |
| ○ PointConv [82]             | 2019 | 61.0 | 66.6 |
| ○ JointPointBased [10]       | 2019 | 69.2 | 63.4 |
| ○ KPConv [72]                | 2019 | 69.2 | 68.6 |
| ○ PointASNL [91]             | 2020 | 63.5 | 66.6 |
| ○ SegGCN [42]                | 2020 | -    | 58.9 |
| ○ RandLA-Net [30]            | 2020 | -    | 64.5 |
| ○ JSENet [31]                | 2020 | -    | 69.9 |
| ○ FusionNet [98]             | 2020 | -    | 68.8 |
| ○ FastPointTransformer [55]  | 2022 | 72.4 | -    |
| ○ StratifiedTransformer [38] | 2022 | 74.3 | 73.7 |
| ○ PointNeXt [60]             | 2022 | 71.5 | 71.2 |
| ○ LargeKernel3D [8]          | 2023 | 73.5 | 73.9 |
| ○ PointMetaBase [45]         | 2023 | 72.8 | 71.4 |
| ○ PointConvFormer [83]       | 2023 | 74.5 | 74.9 |
| ○ OctFormer [77]             | 2023 | 75.7 | 76.6 |
| ○ Swin3D [95]                | 2023 | 75.5 | -    |
| ● + Supervised [95]          | 2023 | 76.7 | 77.9 |
| ○ MinkUNet [12]              | 2019 | 72.2 | 73.6 |
| ● + PC [88]                  | 2020 | 74.1 | -    |
| ● + CSC [28]                 | 2021 | 73.8 | -    |
| ● + MSC [86]                 | 2023 | 75.5 | -    |
| ● + PPT [85]                 | 2023 | 76.4 | 76.6 |
| ○ PTv1 [100]                 | 2021 | 70.6 | -    |
| ○ PTv2 [84]                  | 2022 | 75.4 | 74.2 |
| ○ PTv3 (Ours)                | 2023 | 77.5 | 77.9 |

# Today's Learning Objectives

Be able to answer:

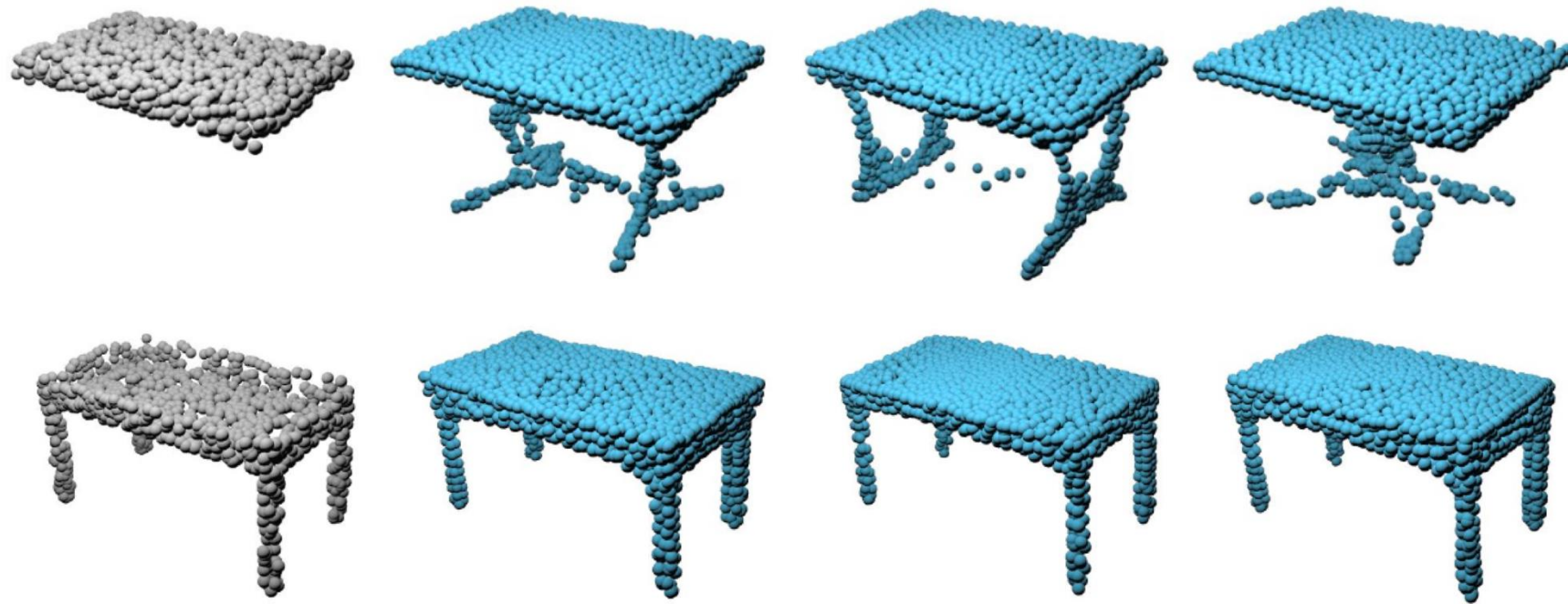
- ~~What is a point cloud?~~
- ~~What is a sparse convolutional network?~~
- ~~What is point convolution?~~
- ~~What is a point transformer?~~
- ~~What are some applications of point cloud networks?~~



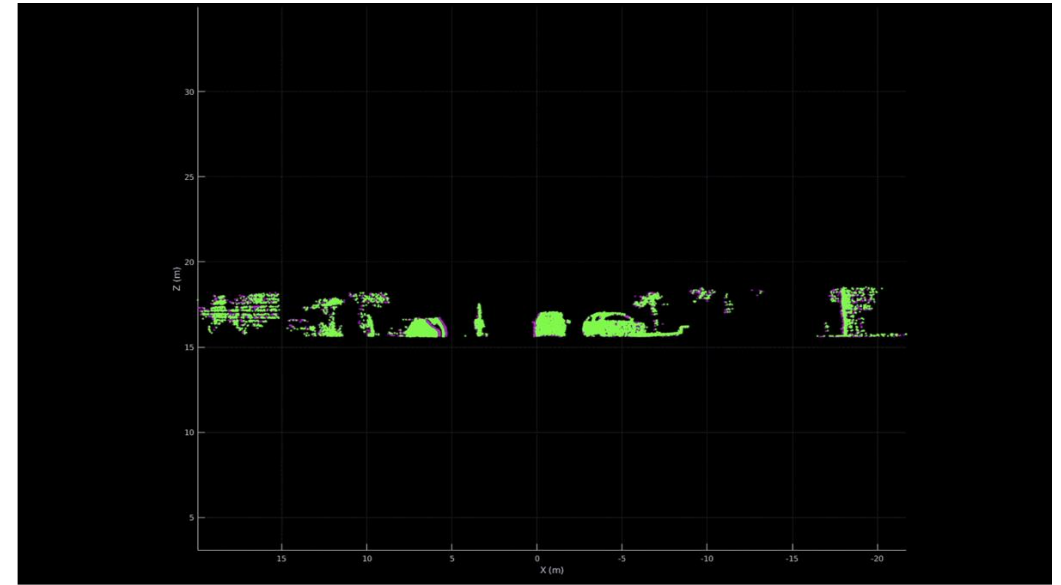
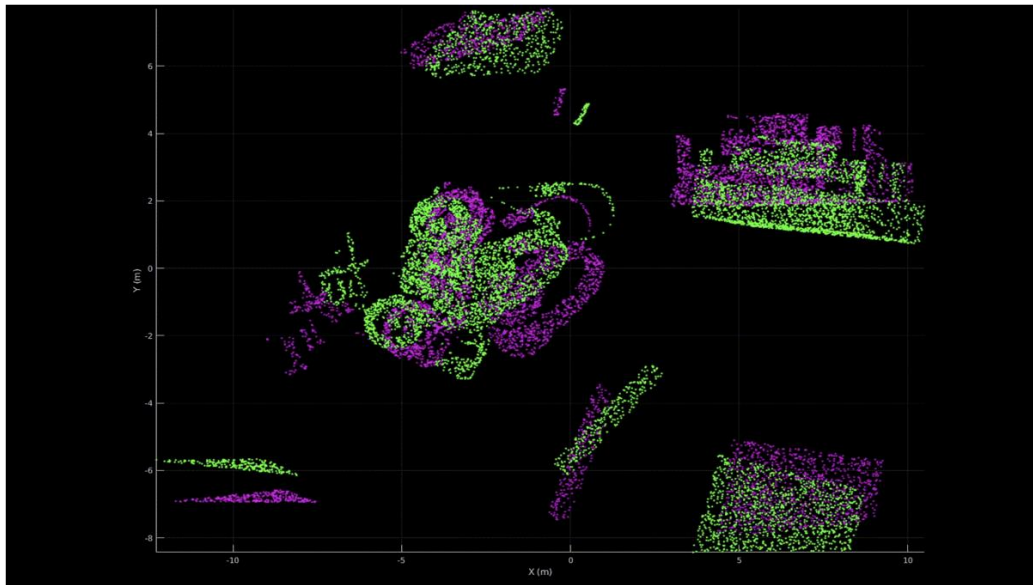


# Point Cloud Applications: Point Cloud Completion

Observe a partial point cloud, run encoder-decoder to complete to a complete point cloud



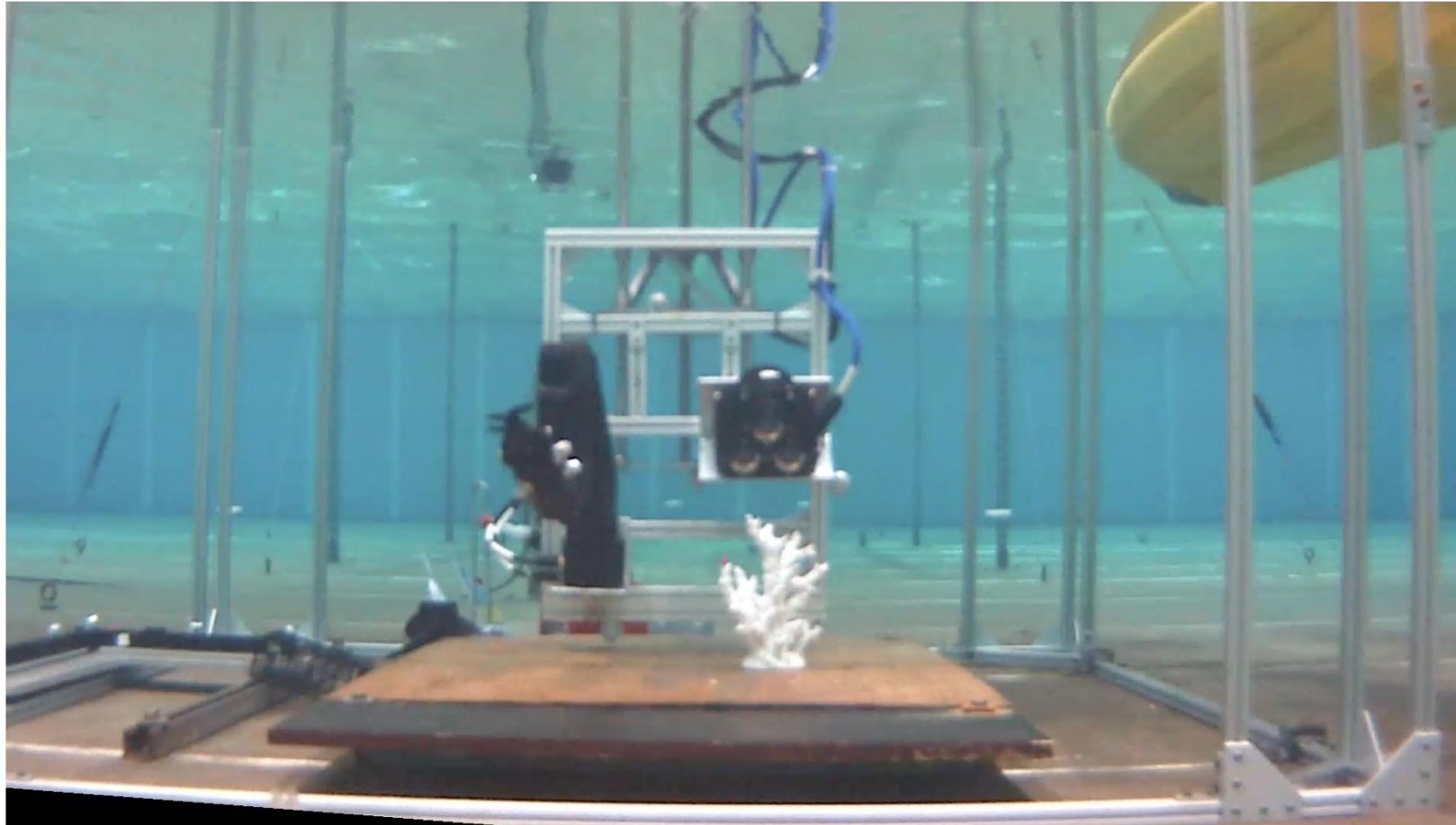
Estimate the correspondence between two 3D point clouds  
each point is allowed to have a different 3D motion





# Point Cloud Applications: Grasp Synthesis

Finding the best point to grasp by running the point cloud through a network and predicting at all points graspable/not graspable



Tim Player, Dongsik Chang, Li Fuxin, Geoffrey Hollinger. [Real-Time Generative Grasping with Spatio-Temporal Sparse Convolution](#). ICRA, 2023



Use a point cloud backbone to predict the motion of every point given initial velocity

## Video Demo